

Chapter 5

8086 Instructions

Learning Outcomes

After completion of this chapter the students will know

- ❖ Different assembly instructions and their usage
- ❖ Use of assembly instructions in assembly programs

5.1 Assembly Instructions

Assembly instructions are special keywords for accessing the internal resources of a microprocessor. They are low level mnemonics usually followed by operands instructing the processor what to do.

5.2 8086 Assembly Instructions

As stated earlier, 8086 have 117 different instructions that allow the user to write programs efficiently. The 8086 instructions can be classified into eight different categories.

1. Data movement instructions (MOV, LEA, LES, PUSH, POP, PUSHF, POPF)
2. Data conversions (CBW, CWD, XLAT)
3. Arithmetic instructions (ADD, INC, SUB, DEC, CMP, NEG, MUL, IMUL, DIV, IDIV)
4. Logical shift, rotate, and bit instructions (AND, OR, XOR, NOT, SHL, SHR, RCL, RCR)
5. I/O instructions (IN, OUT)
6. String instructions (MOVS, STOS, LODS)
7. Program flow control instructions (JMP, CALL, RET) and conditional jumps
8. Miscellaneous instructions (CLC, STC, CMC, STD, CLD)

The following sections describe all the instructions in these groups and how they operate.

5.3 Data Movement Instructions

The data movement instructions copy values from one location to another. These instructions include MOV, XCHG, LDS, LEA, LES, LFS, LGS, LSS, PUSH, PUSHA, PUSHD, PUSHF, PUSHFD, POP, POPA, POPAD, POPF, POPFD, LAHF, and SAHF.

5.3.1 The MOV Instruction

The MOV instruction is very common in programming and takes several different forms. It allows data movement between different resources except "memory to memory".

Syntax: MOV Destination, Source

- MOV REG, REG
- MOV MEM, REG
- MOV REG, MEM
- MOV MEM, IMMEDIATE DATA

```

MOV    REG, IMMEDIATE DATA
MOV    AX/AL, MEM
MOV    MEM, AX/AL
MOV    SEGREG, MEM16
MOV    SEGREG, REG16
MOV    MEM16, SEGREG
MOV    REG16, SEGREG

```

Example 5:1

Move 40H into ES. But data movement from immediate data to segment registers is not permitted.

```

MOV    AX, 40H
MOV    ES, AX

```

5.3.2 The XCHG Instruction

The XCHG (exchange) instruction swaps two values. The general form is

```
XCHG  OPERAND1, OPERAND2
```

There are three specific forms of this instruction on the 8086:

```

XCHG  REG, MEM
XCHG  REG, REG
XCHG  AX, REG16

```

The XCHG instruction does not modify any flags.

5.3.3 The LDS, LES Instructions

The LDS, LES instructions let you load a 16 bit general purpose register and segment register pair with a single instruction. The general form is

These instructions take the specific forms:

```

LDS    REG16, MEM32
LES    REG16, MEM32

```

5.3.4 The LEA Instruction

The LEA (Load Effective Address) instruction is another instruction used to prepare pointer values. The lea instruction takes the form:

```
LEA    REG16, MEM
```

Example 5:2

```
LEA    AX, [BX]
```

5.3.5 The PUSH and POP Instructions

The 8086 PUSH and POP instructions manipulate data on the 8086's hardware stack. There are 19 varieties of the push and pop instructions, they are

```

PUSH    REG16
POP     REG16
PUSH    SEGREG
POP     SEGREG      (EXCEPT CS)
PUSH    MEMORY

```

POP MEMORY
 PUSHF
 POPF

5.3.6 The LAHF and SAHF Instructions

The LAHF instruction loads AH from flags and SAHF stores AH into flags.

5.4 Data conversions

CBW

This instruction converts byte to word.

CWD

This instruction converts word to double word.

XLAT

This instruction translates a byte in the AL register with a byte from a lookup table in memory. It does the following:

$TEMP \leftarrow AL + BX$
 $AL \leftarrow DS:[TEMP]$

That is, BX points at a table in the current data segment. XLAT replaces the value in AL with the byte at the offset originally in AL. The XLAT instruction takes the form:

XLAT

Example

If AL contains four, XLAT replaces the value in AL with the fifth item (offset four) within the table pointed at by DS:BX.

5.5 Arithmetic Instructions

The 8086 provides many arithmetic operations: addition, subtraction, negation, multiplication, division/modulo (remainder), and comparing two values. The instructions that handle these operations are ADD, ADC, SUB, SBB, MUL, IMUL, DIV, IDIV, CMP, NEG, INC and DEC.

The generic forms for these instructions are

ADD	DEST, SRC	$DEST \leftarrow DEST + SRC$
ADC	DEST, SRC	$DEST \leftarrow DEST + SRC + CARRY$
SUB	DEST, SRC	$DEST \leftarrow DEST - SRC$
SBB	DEST, SRC	$DEST \leftarrow DEST - SRC - CARRY$
MUL	SRC	$ACC \leftarrow ACC * SRC$
IMUL	SRC	$ACC \leftarrow ACC * SRC$
DIV	SRC	
IDIV	SRC	
CMP	DEST, SRC	$DEST - SRC$ (AND SET FLAGS)
NEG	DEST	$DEST \leftarrow - DEST$
INC	DEST	$DEST \leftarrow DEST + 1$
DEC	DEST	$DEST \leftarrow DEST - 1$

5.5.1 ADC, ADD

These instructions take the forms:

```
ADD    REG, REG
ADD    REG, MEM
ADD    MEM, REG
ADD    REG, IMMEDIATE DATA
ADD    MEM, IMMEDIATE DATA
ADD    EAX/AX/AL, IMMEDIATE DATA
```

ADC forms are identical to ADD.

ADC is normally used along with ADD instruction when the addition results a carry.

Examples:

```
ADD AL, 75H          ; [AL] ← [AL] + 75H
ADC CL, BL           ; [CL] ← [CL] + [BL] + CF
ADD DX, [SI]         ; [DX] ← [DX] + [[DS]*16 + [SI]]
ADC AL, COST [BX]    ; [AL] ← [AL] + [[DS]*16 + [BX+COST]] + CF
```

5.5.2 SBB, SUB

The SUB (subtract) and SBB (subtract with borrow) have syntaxes similar to that of the ADD instructions:

```
SUB    REG, REG
SUB    REG, MEM
SUB    MEM, REG
SUB    REG, IMMEDIATE DATA
SUB    MEM, IMMEDIATE DATA
SUB    EAX/AX/AL, IMMEDIATE DATA
```

SBB forms are identical to SUB.

SBB is normally used along with SUB instructions. When a subtraction operation needs a borrow, it is taken care of by the SBB instruction.

Example 5:3

```
SUB CX, BX          ; [CX] ← [CX] - [BX]
SBB CH, AL          ; [CH] ← [CH] - [AL] - CF
SUB AX, 4563H       ; [AX] ← [AX] - 4563H
SBB BX, [4563H]     ; [AX] ← [BX] - [[DS]*16 + 4563H]
```

5.5.3 MUL, IMUL

The MUL instruction multiplies unsigned bytes or words. The IMUL instruction multiplies signed bytes or words.

Example 5:4

```
MUL BH              ; [AX] ← [AL]*[BH]
MUL CX              ; [AX]*[CX] → [AX] - Low word, [DX] - High word
MUL BYTE PTR [BX]  ; [AX] ← [AL]*[[DS]*16 + [BX]]
IMUL BH             ; [AX] ← [AL]*[BH] where [AL] and [BH] are signed bytes
```

IMUL CX ; [AX]*[CX] → [AX] – Low word, [DX] – High word

5.5.4 The Division Instructions: DIV, IDIV

The 8086 divide instructions perform a 16/8 division. These instructions take the form:

DIV REG FOR UNSIGNED DIVISION

DIV MEM

IDIV REG FOR SIGNED DIVISION

IDIV MEM

Example 5:5

```
DIV BL ; [AX]/[BL]
        ; [AL] ← Quotient, [AH] ← Remainder
DIV CX ; Double word [DX][AX] / [CX]
        ; [AX] ← Quotient, [DX] ← Remainder
IDIV BL ; [AX]/[BL]
        ; [AL] ← Quotient, [AH] ← Remainder
IDIV CX ; Double word [DX][AX] / [CX]
        ; [AX] ← Quotient, [DX] ← Remainder
IDIV BYTE PTR [BX] ; [AX]/BYTE at offset [BX] in DS
        ; [AX]/[[DS]*16+[BX]]
```

5.5.5 INC

The INC instruction increments the operand source by 1. It takes the forms:

INC REG

INC MEM

INC REG16

5.5.6 DEC

The DEC instruction decrements the operand source by 1. It takes the forms:

DEC REG

DEC MEM

DEC REG16

5.6 ASCII adjustments instructions

During ASCII arithmetic, adjustments are needed as in ASCII codes 0-9 are represented by 30H-39H. ASCII adjustments give decimal results in unpacked BCD form. In unpacked BCD form a BCD digit is represented by one byte. For example, decimal 34 in unpacked BCD format is 00000011 00000100. Unpacked ASCII numbers are usually sent by serial communication devices.

5.6.1 AAA – ASCII adjust for addition

Example 5:6

Add two ASCII numbers, ASCII 5 → AL, ASCII 9 → BL and store the result in AL.

```
ADD AL, BL ; AL = 6EH
```

```
AAA ; AL = 00000100 (decimal 4), Carry = 1
```

5.6.2 AAD – ASCII adjust for division

Used for dividing unpacked BCD numbers. AAD converts the unpacked BCD into binary form.

Example 5:7

Let AX contains decimal 67 in unpacked BCD form ($AX \leftarrow 0607$). Divide it with 09H contained in CH register.

```
AAD          ; Adjust the unpacked BCD number to binary before division
              ;  $AX \leftarrow 0043 = 43H = 67$  decimal
DIV CH       ; Divide AX by unpacked BCD in CH
              ;  $AL \leftarrow$  Quotient = 07 unpacked BCD
              ;  $AH \leftarrow$  Remainder = 04 unpacked BCD
```

5.6.3 AAM – ASCII adjust for Multiplication

After multiplication, result is in binary form. AAM unpacks the binary result in unpacked BCD form.

Example 5:8

The contents of AL = 5 and BH = 9. Multiply [AL] with [BH] and store the result in AX in unpacked BCD format.

```
MUL BH      ;  $[AX] \leftarrow [AL] * [BH] = 002DH$ 
AAM         ;  $[AX] \leftarrow 0405$ , unpacked BCD for 45
```

5.6.4 AAS – ASCII adjust for Subtraction

After subtracting two ASCII codes, AAS make sure that the result is in correct unpacked BCD format.

Example 5:9

The contents of AL = ASCII 5 and BL = ASCII 9. Subtract [BL] from [AL] and adjust the result in unpacked BCD format.

```
SUB AL, BL  ;  $[AL] \leftarrow [AL] - [BL] = FCH = -4$  in 2's complement
              ; CF = 1
AAS         ;  $[AL] \leftarrow 04$ , unpacked BCD for 4
              ; CF = 1
```

5.6.5 The CMP Instruction

The CMP (compare) instruction is identical to the SUB instruction with one crucial difference - it does not store the difference back into the destination operand. The syntax for the CMP instruction is very similar to SUB, the generic form is

```
CMP  DEST, SRC
```

The specific forms are

```
CMP  REG, REG
```

```
CMP  REG, MEM
```

```
CMP  MEM, REG
```

```
CMP  REG, IMMEDIATE DATA
```

```
CMP  MEM, IMMEDIATE DATA
```

```
CMP  EAX/AX/AL, IMMEDIATE DATA
```

The CMP instruction updates the 8086's flags according to the result of the subtraction operation (DEST - SRC). You can test the result of the comparison by checking the appropriate flags in the flags register.

5.7 Logical, Shift, Rotate and Bit Instructions

The 8086 provides five logical instructions, four rotate instructions, and three shift instructions. The logical instructions are AND, OR, XOR, TEST and NOT, the rotate instructions are ROR, ROL, RCR and RCL; the shift instructions are SHL/SAL, SHR and SAR. These instructions can manipulate bits, convert values, do logical operations, pack and unpack data, and do arithmetic operations.

5.7.1 The Logical Instructions: AND, OR, XOR, and NOT

The 8086 logical instructions operate on a bit-by-bit basis. The AND, NOT, OR and XOR instructions do the following:

AND	DEST, SOURCE	DEST ← DEST AND SOURCE
OR	DEST, SOURCE	DEST ← DEST OR SOURCE
XOR	DEST, SOURCE	DEST ← DEST XOR SOURCE
NOT	DEST	DEST ← NOT DEST

The specific variations are

AND	REG, REG
AND	MEM, REG
AND	REG, MEM
AND	REG, IMMEDIATE DATA
AND	MEM, IMMEDIATE DATA
AND	EAX/AX/AL, IMMEDIATE DATA

OR uses the same formats as AND

XOR uses the same formats as AND

NOT	REGISTER
NOT	MEM

5.7.2 The Shift Instructions: SHL/SAL, SHR, SAR

The 8086 supports three different shift instructions SHL (shift left), SAL (shift arithmetic left), SHR (shift right), and SAR (shift arithmetic right). The shift instructions move bits around in a register or memory location. The general format for a shift instruction is

SHL	DEST, COUNT
SAL	DEST, COUNT
SHR	DEST, COUNT
SAR	DEST, COUNT

DEST is the value to shift and COUNT specifies the number of bit positions to shift. For example, the SHL instruction shifts the bits in the destination operand to the left the number of bit positions specified by the COUNT operand.

The specific forms for these instructions are

SHL	REG, 1
SHL	MEM, 1
SHL	REG, CL

SHL MEM, CL

SAL is a synonym for SHL and uses the same formats.

SHR uses the same formats as SHL.

SAR uses the same formats as SHL.

SHL/SAL

The SHL and SAL mnemonics are synonyms. They represent the same instruction and use identical binary encodings. These instructions move each bit in the destination operand one bit position to the left the number of times specified by the count operand. Zeros fill vacated positions at the LSB; the MSB shifts into the carry flag:

SAR

The SAR instruction shifts all the bits in the destination operand to the right one bit, the MSB remains unchanged. The LSB goes to carry flag. The result is signed divide by 2.

SHR

The SHR instruction shifts all the bits in the destination operand to the right one bit shifting a zero into the MSB, the LSB shifts to Carry. The result is unsigned divide by 2.

5.7.3 The Rotate Instructions: RCL, RCR, ROL, and ROR

The rotate instructions shift the bits around, just like the shift instructions, except the bits shifted out of the operand by the rotate instructions re-circulate through the operand. They include RCL (rotate through carry left), RCR (rotate through carry right), ROL (rotate left), and ROR (rotate right). These instructions all take the forms:

RCL DEST, COUNT

ROL DEST, COUNT

RCR DEST, COUNT

ROR DEST, COUNT

The specific forms are

RCL REG, 1

RCL MEM, 1

RCL REG, CL

RCL MEM, CL

ROL uses the same formats as RCL.

RCR uses the same formats as RCL.

ROR uses the same formats as RCL.

RCL

The RCL (rotate through carry left), as its name implies, rotates bits to the left, through the carry flag, and back into bit zero on the right.

RCR

The RCR (rotate through carry right) instruction is the complement to the RCL instruction. It shifts its bits right through the carry flag and back into the MSB.

ROL

The ROL instruction is similar to the RCL instruction in that it rotates its operand to the left the specified number of bits. The major difference is that ROL shifts its operand's MSB, rather than the carry, into bit zero. ROL also copies the output of the MSB into the carry flag.

ROR

The ROR instruction relates to the RCR instruction in much the same way that the ROL instruction relates to RCL. That is, it is almost the same operation other than the source of the input bit to the operand. Rather than shifting the previous carry flag into the MSB of the destination operation, ROR shifts bit zero into the MSB.

5.7.4 TEST Instruction

The TEST instruction logically AND operands to update flags. The basic form is

```
TEST  DEST, SOURCE
```

The specific forms are

```
TEST  REG, REG
```

```
TEST  REG, MEM
```

```
TEST  REG, IMM
```

```
TEST  MEM, IMM
```

```
TEST  EAX/AX/AL, IMM
```

5.8 8086 Instructions A to Z

The 8086 instructions in details are listed alphabetically with examples. The students are advised to test the sample programs on 8086 emulator.

AAA ASCII Adjust after Addition.

Corrects result in AH and AL into unpacked BCD format, generally needed after addition of ASCII numbers.

Example 5:10

```
MOV AX, 15 ; AH = 00, AL = 0Fh
```

```
AAA ; AH = 01, AL = 05 (Unpacked BCD)
```

AAD ASCII Adjust before Division.

Converts unpacked BCD into packed BCD needed for division.

Example 5:11

```
MOV AX, 0105h ; AH = 01, AL = 05
```

```
AAD ; AH = 00, AL = 0Fh (15)
```

AAM ASCII Adjust after Multiplication.

Converts (packed) BCD number into unpacked BCD format

Example 5:12

```
MOV AL, 15 ; AL = 0Fh
```

```
AAM ; AH = 01, AL = 05
```

AAS ASCII Adjust after Subtraction.

Corrects result in AH and AL after subtraction when working with BCD values.

Example 5:13

```
MOV AL, 32H ; ASCII code for BCD = 2
MOV BL, 35H ; ASCII code for BCD = 5
SUB AL, BL   ; AL = FDH
AAS          ; AL = 03, CF = 1
```

ADC {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}

Adds operand1 to operand2 and carry

Example 5:14

```
STC          ; set CF = 1
MOV AL, 5     ; AL = 5
ADC AL, 1     ; AL = 7
```

ADD {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}

Adds operand1 and operand2

Example 5:15

```
MOV AL, 5     ; AL = 5
ADD AL, -3    ; AL = 2
```

AND {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}

Logical AND between all bits of two operands

Result is stored in operand1.

Example 5:16

```
MOV AL, 'a'   ; AL = 01100001b
AND AL, 11011111b ; AL = 01000001b ('A')
```

CALL {procedure name or Label or 4-byte address}

Transfers control to procedure, return address (IP) is pushed to stack. *4-byte address* may be entered in this form: 1234h:5678h, first value is a segment second value is an offset (this is a far call, so CS is also pushed to stack).

Example 5:17

```
ORG 100H
CALL PR1
ADD AX, 1
RET          ; return to OS.
```

PR1 PROC ; procedure declaration.

MOV AX, 1234h

RET ; return to caller.

PR1 ENDP

CBW Convert byte into word.

Example 5:18

```
MOV AX, 0     ; AH = 0, AL = 0
MOV AL, -5    ; AX = 000FBh (251)
CBW           ; AX = 0FFFBh (-5)
```

CLC Clear Carry flag.
CF = 0

CLD Clear Direction flag.
SI and DI will be incremented by chain instructions: CMPSB, CMPSW, LODSB, LODSW, MOVS, MOVSW, STOSB, STOSW.

CLI Clear Interrupt enable flag.
This disables hardware interrupts.

CMC Complement Carry flag.
Inverts value of CF.

CMP {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}
Compare operand1 and operand2 (operand1-operand2)
Result is not stored anywhere, flags are set (OF, SF, ZF, AF, PF, CF) according to result.

Example 5:19

```
MOV AL, 5
MOV BL, 5
CMP AL, BL ; AL = 5, ZF = 1 (so equal!)
```

CWD Convert Word to Double word
if high bit of AX = 1 then: DX = 65535 (0FFFFh) else DX = 0

Example 5:20

```
MOV DX, 0 ; DX = 0
MOV AX, 0 ; AX = 0
MOV AX, -5 ; DX AX = 00000h:0FFFBh
CWD ; DX AX = 0FFFFh:0FFFBh
```

DAA Decimal Adjust after Addition.
Corrects the result of addition of two packed BCD values [if low nibble of AL > 9 or AF = 1 then: AL = AL + 6, AF = 1. if AL > 9Fh or CF = 1 then: AL = AL + 60h, CF = 1]

Example 5:21

```
MOV AL, 0Fh ; AL = 0Fh (15)
DAA ; AL = 15h
```

DAS Decimal adjust After Subtraction.
Corrects the result of subtraction of two packed BCD values. [if low nibble of AL > 9 or AF = 1 then: AL = AL - 6, AF = 1. if AL > 9Fh or CF = 1 then: AL = AL - 60h, CF = 1]

Example 5:22

```
MOV AL, 0FFh ; AL = 0FFh (-1)
DAS ; AL = 99h, CF = 1
```

DEC {REG or memory}
Decrement [operand = operand - 1]

Example 5:23

```
MOV AL, 255 ; AL = 0FFh (255 or -1)
DEC AL      ; AL = 0FEh (254 or -2)
```

DIV { REG or Memory} Unsigned divide

When operand is a **byte**: AL = AX / operand, AH = remainder (modulus). when operand is a **word**: AX = (DX AX) / operand, DX = remainder (modulus)

Example 5:24

```
MOV AX, 203 ; AX = 00CBh
MOV BL, 4
DIV BL      ; AL = 50 (32h), AH = 3
```

HLT Halt the System.

Example 5:25

```
MOV AX, 5
HLT
```

IDIV {REG or memory} Signed divide

When operand is a **byte**: AL = AX / operand, AH = remainder (modulus). when operand is a **word**: AX = (DX AX) / operand, DX = remainder (modulus).

Example 5:26

```
MOV AX, -203 ; AX = 0FF35h
MOV BL, 4
IDIV BL      ; AL = -50 (0CEh), AH = -3 (0FDh)
```

IMUL {REG or memory} Signed multiply

When operand is a **byte**: AX = AL * operand. when operand is a **word**: (DX AX) = AX * operand.

Example 5:27

```
MOV AL, -2
MOV BL, -4
IMUL BL      ; AX = 8
```

IN {AL, im.byte or AL, DX or AX, im.byte or AX, DX}

Input from port into AL or AX.

Second operand is a port number. If required to access port number over 255 - **DX** register should be used.

Example 5:28

```
IN AX, 4 ; get status of traffic lights.
IN AL, 7 ; get status of stepper-motor.
```

INC {REG or memory} Increment

operand = operand + 1

Example 5:29

```
MOV AL, 4
INC AL      ; AL = 5
```

INT {immediate byte} Software interrupt

Interrupt numbered by immediate byte (0..255) [Push to stack: flags register, CS, IP and set IF = 0. Transfer control to interrupt procedure].

Example 5:30

```
MOV AH, 0Eh ; teletype.  
MOV AL, 'A'  
INT 10h    ; BIOS interrupt.
```

INTO Interrupt on overflow

Interrupt 4 if Overflow flag is 1 [if OF = 1 then INT 4].

Example 5:31

```
; -5 - 127 = -132 (not in -128..127)  
; the result of SUB is wrong (124),  
; so OF = 1 is set:  
MOV AL, -5  
SUB AL, 127 ; AL = 7Ch (124)  
INTO        ; process error.
```

IRET Interrupt Return

Pop from stack: IP, CS, flags register.

JA {label} Short jump if above

Short Jump if first operand is Above second operand (as set by CMP instruction).

Example 5:32

```
IN AL, ADC  
CMP AL, 50  
JA LABEL1  
JMP LABEL2  
LABEL1: OUT 01H, PORTA  
LABEL2: OUT 02H, PORTA
```

JAE {label} Short jump if above or equal

Short Jump if first operand is Above or Equal to second operand (as set by CMP instruction).

Example 5:33

```
IN AL, ADC  
CMP AL, 50  
JAE LABEL1  
JMP LABEL2  
LABEL1: OUT 01H, PORTA  
LABEL2: OUT 02H, PORTA
```

JB {label} Short jump if below

Short Jump if first operand is Below second operand (as set by CMP instruction).

Example 5:34

```
IN AL, ADC  
CMP AL, 50  
JB LABEL2  
JMP LABEL1  
LABEL1: OUT 01H, PORTA  
LABEL2: OUT 02H, PORTA
```

JBE {label} Short jump if below or equal
Short Jump if first operand is Below or Equal to second operand (as set by CMP instruction).

Example 5:35

```
        IN AL, ADC
        CMP AL, 50
        JB LABEL2
        JMP LABEL1
LABEL1:  OUT 01H, PORTA
LABEL2:  OUT 02H, PORTA
```

JC {label} Short jump if carry
Short Jump if Carry flag is set to 1.

Example 5:36

```
        XOR AX, AX
        MOV AL, 255
        ADD AL, 1
        JC label1
        JMP exit
label1:
        ADC AH,0
exit:
        HLT
```

JE {label} Short jump if equal
Short jump if first operand is equal to second operand (as set by CMP instruction). If ZF = 1 then jump

Example 5:37

```
        MOV AL, 5
        CMP AL, 5
        JE label1
        JMP exit
label1:

exit:
```

JG {label} Short jump if greater
Short Jump if first operand is Greater then second operand (as set by CMP instruction). If (ZF = 0) and (SF = OF) then jump

Example 5:38

```
        MOV AL, 5
        CMP AL, -5
        JG label1
        JMP exit
label1:
exit:
```

JGE {label} Short jump if greater than or equal

Short jump if first operand is greater or equal to second operand (as set by CMP instruction).

Example 5:39

```
MOV AL, 2
CMP AL, -5
JGE label1
JMP exit
label1:
exit:
```

JL {label} Short jump if less
Short Jump if first operand is Less then second operand (as set by CMP instruction).

Example 5:40

```
MOV AL, -2
CMP AL, 5
JL label1
JMP exit
label1:
exit:
```

JLE {label} Short jump if less or equal
Short jump if first operand is less or equal to second operand (as set by CMP instruction).

Example 5:41

```
MOV AL, -2
CMP AL, 5
JLE label1
JMP exit
label1:
exit:
```

JMP {label or 4-byte address} Unconditional jump (long jump)
Transfers control to another part of the program. *4-byte address* may be entered in this form: 1234h:5678h, first value is a segment second value is an offset.

Example 5:42

```
MOV AL, 5
JMP label1 ; jump over 2 lines!
MOV AL, 0
MOV AH, 0
label1:
```

JNA {label} Short jump if not above
Short jump if first operand is not above second operand (as set by CMP instruction).
If CF = 1 or ZF = 1 then jump

Example 5:43

```
MOV AL, 2
CMP AL, 5
JNA label1
```

```
JMP exit
label1:
exit:
```

JNAE {label} Short jump if not above or equal
Short jump if first operand is not above and not equal to second operand (as set by CMP instruction). If CF = 1 then jump

Example 5:44

```
MOV AL, 2
CMP AL, 5
JNAE label1
JMP exit
label1:
exit:
```

JNB {label} Short jump if not below
Short Jump if first operand is Not Below second operand (as set by CMP instruction). If CF = 0 then jump

Example 5:45

```
MOV AL, 7
CMP AL, 5
JNB label1
JMP exit
label1:
exit:
```

JNBE {label} Short jump if not below and equal
Short Jump if first operand is not below and not equal to second operand (as set by CMP instruction). If (CF = 0) and (ZF = 0) then jump

Example 5:46

```
MOV AL, 7
CMP AL, 5
JNBE label1
JMP exit
label1:
exit:
```

JNC {label} Shot jump if not carry
Short Jump if Carry flag is set to 0. If CF = 0 then jump

Example 5:47

```
MOV AL, 2
ADD AL, 3
JNC label1
JMP exit
label1:
exit:
```

JNE {label} Short jump if not equal

Short Jump if first operand is Not Equal to second operand (as set by CMP instruction). If ZF = 0 then jump

Example 5:48

```
MOV AL, 2
CMP AL, 3
JNE label1
JMP exit
label1:
exit:
```

JNG {label} Short jump if not greater

Short Jump if first operand is Not Greater then second operand (as set by CMP instruction). If (ZF = 1) and (SF <> OF) then jump

Example 5:49

```
MOV AL, 2
CMP AL, 3
JNG label1
JMP exit
label1:
exit:
```

JNGE {label} Short jump if not greater and not equal

Short Jump if first operand is Not Greater and Not Equal to second operand (as set by CMP instruction).

Example 5:50

```
MOV AL, 2
CMP AL, 3
JNGE label1
JMP exit
label1:
exit:
```

JNL {label} Short jump if not less

Short Jump if first operand is Not Less then second operand (as set by CMP instruction). If SF = OF then jump

Example 5:51

```
MOV AL, 2
CMP AL, -3
JNL label1
JMP exit
label1:
exit:
```

JNLE {label} Short jump if not less and not equal

Short jump if first operand is not less and not equal to second operand (as set by CMP instruction). If (SF = OF) and (ZF = 0) then jump

Example 5:52

```
MOV AL, 2
```

```

CMP AL, -3
JNLE label1
JMP exit
label1:
exit:

```

JNO {label} Short jump if no overflow
 If OF = 0 then jump

Example 5:53

```

; -5 - 2 = -7 (inside -128..127)
; the result of SUB is correct,
; so OF = 0:
MOV AL, -5
SUB AL, 2 ; AL = 0F9h (-7)
JNO label1
JMP exit
label1:
exit:

```

JNP {label} Short jump if no parity (i.e., parity odd)
 Only 8 low bits of result are checked. Set by CMP, SUB, ADD, TEST, AND, OR, XOR instructions. If PF = 0 then jump

Example 5:54

```

MOV AL, 00000111b ; AL = 7
OR AL, 0 ; just set flags.
JNP label1
JMP exit
label1:
exit:

```

JNS {label} Short jump if not signed (if positive).
 Set by CMP, SUB, ADD, TEST, AND, OR, XOR instructions. If SF = 0 then jump

Example 5:55

```

MOV AL, 00000111b ; AL = 7
OR AL, 0 ; just set flags.
JNS label1
JMP exit
label1:
exit:

```

JNZ {label} Short Jump if Not Zero (not equal).
 Set by CMP, SUB, ADD, TEST, AND, OR, XOR instructions. If ZF = 0 then jump

Example 5:56

```

MOV AL, 00000111b ; AL = 7
OR AL, 0 ; just set flags.
JNZ label1
JMP exit
label1:

```

exit:

JO {label}
Short Jump if Overflow. If OF = 1 then jump

Example 5:57

```
; -5 - 127 = -132 (not in -128..127)
; the result of SUB is wrong (124),
; so OF = 1 is set:
MOV AL, -5
SUB AL, 127 ; AL = 7Ch (124)
JO label1
JMP exit
label1:
exit:
```

JP {label} Short jump if parity (even).
Only 8 low bits of result are checked. Set by CMP, SUB, ADD, TEST, AND, OR, XOR instructions. If PF = 1 then jump

Example 5:58

```
MOV AL, 00000101b ; AL = 5
OR AL, 0 ; just set flags.
JP label1
JMP exit
label1:
exit:
```

JPE {label} Short jump if parity even.
Only 8 low bits of result are checked. Set by CMP, SUB, ADD, TEST, AND, OR, XOR instructions. If PF = 1 then jump

Example 5:59

```
MOV AL, 00000101b ; AL = 5
OR AL, 0 ; just set flags.
JPE label1
JMP exit
label1:
exit:
```

JPO {label} Short jump if parity odd.
Only 8 low bits of result are checked. Set by CMP, SUB, ADD, TEST, AND, OR, XOR instructions. If PF = 0 then jump

Example 5:60

```
MOV AL, 00000111b ; AL = 7
OR AL, 0 ; just set flags.
JPO label1
JMP exit
label1:
exit:
```

JS {label} Short jump if signed (if negative).

Set by CMP, SUB, ADD, TEST, AND, OR, XOR instructions. If SF = 1 then jump

Example 5:61

```
MOV AL, 10000000b ; AL = -128
OR  AL, 0          ; just set flags.
JS  label1
JMP exit
label1:
exit:
```

JZ {label} Short jump if zero (equal).

Set by CMP, SUB, ADD, TEST, AND, OR, XOR instructions. If ZF = 1 then jump

Example 5:62

```
MOV AL, 5
CMP AL, 5
JZ  label1
JMP exit
label1:
exit:
```

LAHF Load AH from 8 low bits of Flags register.
AH = flags register

AH bit: 7 6 5 4 3 2 1 0
[SF] [ZF] [0] [AF] [0] [PF] [1] [CF]
bits 1, 3, 5 are reserved.

LDS {REG, memory} Load memory double word into word register and DS.
REG = first word, DS = second word

Example 5:63

```
LDS AX, M
RET
```

```
M    DW    1234h
      DW    5678h
END
```

AX is set to 1234h, DS is set to 5678h.

LEA {REG, memory} Load Effective Address.
REG = address of memory (offset)

Example 5:64

```
MOV BX, 35h
MOV DI, 12h
LEA SI, [BX+DI] ; SI = 35h + 12h = 47h
```

LDS {REG, memory} Load memory double word into word register and ES.
REG = first word, ES = second word

Example 5:65

```

LES AX, M
RET

M      DW    1234h
      DW    5678h
END

```

AX is set to 1234h, ES is set to 5678h.

```

LOOP {label}      Decrease CX, jump to label if CX not zero.
                  CX = CX - 1, if CX <> 0 then jump else no jump, continue

```

Example 5:66

```

MOV CX, 5
label1:
LOOP label1

```

```

LOOPE {label}      Decrease CX, jump to label if CX not zero and Equal (ZF = 1).
                  CX = CX - 1, if (CX <> 0) and (ZF = 1) then jump else no jump, continue

```

Example 5:67

```

; Loop until result fits into AL alone,
; or 5 times. The result will be over 255
; on third loop (100+100+100),
; so loop will exit.

```

```

MOV AX, 0
MOV CX, 5
label1:
ADD AX, 100
CMP AH, 0
LOOPE label1

```

```

LOOPNE {label}     Decrease CX, jump to label if CX not zero and Not Equal (ZF = 0).
                  CX = CX - 1, if (CX <> 0) and (ZF = 0) then jump else no jump, continue

```

Example 5:68

```

; Loop until '7' is found,
; or 5 times.

```

```

MOV SI, 0
MOV CX, 5
label1:
MOV AL, v1[SI]
INC SI      ; next byte (SI=SI+1).
CMP AL, 7
LOOPNE label1
RET
v1 db 9, 8, 7, 6, 5

```

```

LOOPNZ {label}     Loop if not zero

```

Decrease CX, jump to label if CX not zero and ZF = 0.

CX = CX - 1, if (CX <> 0) and (ZF = 0) then jump else no jump, continue

Example 5:69

```
; Loop until '7' is found,  
; or 5 times.  
MOV SI, 0  
MOV CX, 5  
label1:  
MOV AL, v1[SI]  
INC SI      ; next byte (SI=SI+1).  
CMP AL, 7  
LOOPNZ label1  
RET  
v1 db 9, 8, 7, 6, 5
```

LOOPZ {label} Loop if zero

Decrease CX, jump to label if CX not zero and ZF = 1.

CX = CX - 1, if (CX <> 0) and (ZF = 1) then jump else no jump, continue

Example 5:70

```
; Loop until result fits into AL alone,  
; or 5 times. The result will be over 255  
; on third loop (100+100+100),  
; so loop will exit.  
MOV AX, 0  
MOV CX, 5  
label1:  
ADD AX, 100  
CMP AH, 0  
LOOPZ label1  
RET
```

MOV {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate or SREG, memory or memory, SREG or REG, SREG or SREG, REG}
Copy operand2 to operand1.

The MOV instruction cannot set the value of the CS and IP registers. In case of copying value of one segment register to another segment register (should copy to general register first).

To copy immediate value to a segment register, one should copy to general register first.

Example 5:71

```
MOV AX, 0B800H  
MOV DS, AX      ; copy value of AX to DS.  
MOV CL, 'A'     ; CL = 41h (ASCII code).  
MOV CH, 01011111B  
MOV BX, 15EH  
MOV [BX], CX
```

MUL {REG or memory} Unsigned multiply.
When operand is a **byte**: $AX = AL * \text{operand}$.
When operand is a **word**: $(DX\ AX) = AX * \text{operand}$.

Example 5:72

```
MOV AL, 200 ; AL = 0C8h
MOV BL, 4
MUL BL ; AX = 0320h (800)
RET
CF=OF=0 when high section of the result is zero.
```

NEG {REG or memory} Negate. Makes operand negative (two's complement).

Invert all bits of the operand and then add 1 to inverted operand

Example 5:73

```
MOV AL, 5 ; AL = 05h
NEG AL ; AL = 0FBh (-5)
NEG AL ; AL = 05h (5)
RET
```

NOP No Operation.

Example 5:74

```
; do nothing, 3 times:
NOP
NOP
NOP
RET
```

NOT {REG or memory} Invert each bit of the operand.
If bit is 1 turn it to 0, if bit is 0 turn it to 1.

Example 5:75

```
MOV AL, 00011011b
NOT AL ; AL = 11100100b
RET
```

OR {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}

Logical OR between all bits of two operands. Result is stored in first operand.

Example 5:76

```
MOV AL, 'A' ; AL = 01000001b
OR AL, 00100000b ; AL = 01100001b ('a')
RET
```

OUT {im.byte, AL or vim.byte, AX or DX, AL or DX, AX} Output from AL or AX to port.

First operand is a port number. If required to access port number over 255 - **DX** register should be used.

Example 5:77

```
MOV AX, 0FFFh ; Turn on all
```

OUT 4, AX ; traffic lights.

MOV AL, 100b ; Turn on the third
OUT 7, AL ; magnet of the stepper-motor.

POP {REG or SREG or memory} Get 16 bit value from the stack.
operand = SS:[SP] (top of the stack)
SP = SP + 2

Example 5:78

MOV AX, 1234h
PUSH AX
POP DX ; DX = 1234h
RET

POPF Get flags register from the stack.
Flags = SS:[SP] (top of the stack)
SP = SP + 2

PUSH {REG or SREG or Memory or Immediate}
Store 16 bit value in the stack
SP = SP - 2
SS:[SP] (top of the stack) = operand

Example 5:79

MOV AX, 1234h
PUSH AX
POP DX ; DX = 1234h
RET

PUSHF Store flags register in the stack.

SP = SP - 2
SS:[SP] (top of the stack) = flags

RCL {memory, immediate or REG, immediate or memory, CL or REG, CL}
Rotate operand1 left through Carry Flag. The number of rotates is set by operand2.
When **immediate** is greater than 1, assembler generates several **RCL xx, 1** instructions because 8086 has machine code only for this instruction (the same principle works for all other shift/rotate instructions).
Shift all bits left, the bit that goes off is set to CF and previous value of CF is inserted to the right-most position.

Example 5:80

STC ; set carry (CF=1).
MOV AL, 1Ch ; AL = 00011100b
RCL AL, 1 ; AL = 00111001b, CF=0.
RET
OF=0 if first operand keeps original sign.

RCR {memory, immediate or REG, immediate or memory, CL or REG, CL}
Rotate operand1 right through Carry Flag. The number of rotates is set by operand2.

Shift all bits right, the bit that goes off is set to CF and previous value of CF is inserted to the left-most position.

Example 5:81

```
STC          ; set carry (CF=1).
MOV AL, 1Ch   ; AL = 00011100b
RCR AL, 1     ; AL = 10001110b, CF=0.
RET
OF=0 if first operand keeps original sign.
```

RET Return from near procedure.

Pop from stack: IP, if immediate operand is present: SP = SP + operand

Example 5:82

```
CALL P1
ADD AX, 1
RET          ; return to OS.

P1 PROC      ; procedure declaration.
    MOV AX, 1234H
    RET      ; return to caller.
P1 ENDP
```

ROL {memory, immediate or REG, immediate or memory, CL or REG, CL}
Rotate operand1 left. The number of rotates is set by operand2.

Example 5:83

```
MOV AL, 1Ch   ; AL = 00011100b
ROL AL, 1     ; AL = 00111000b, CF=0.
RET
OF=0 if first operand keeps original sign.
```

ROR {memory, immediate or REG, immediate or memory, CL or REG, CL}
Rotate operand1 right. The number of rotates is set by operand2.

Example 5:84

```
MOV AL, 1Ch   ; AL = 00011100b
ROR AL, 1     ; AL = 00001110b, CF=0.
RET
OF=0 if first operand keeps original sign.
```

SAHF Store AH register into low 8 bits of Flags register.
Flags register = AH

AH bit: 7 6 5 4 3 2 1 0
[SF] [ZF] [0] [AF] [0] [PF] [1] [CF]
bits 1, 3, 5 are reserved.

SAL {memory, immediate or REG, immediate or memory, CL or REG, CL}
Shift Arithmetic operand1 Left. The number of shifts is set by operand2.

Example 5:85

```
MOV AL, 0E0h   ; AL = 11100000b
```

SAL AL, 1 ; AL = 11000000b, CF=1.
RET
OF=0 if first operand keeps original sign.

SAR {memory, immediate or REG, immediate or memory, CL or REG, CL}
Shift Arithmetic operand1 Right. The number of shifts is set by operand2.

Example 5:86

MOV AL, 0E0h ; AL = 11100000b
SAR AL, 1 ; AL = 11110000b, CF=0.

MOV BL, 4Ch ; BL = 01001100b
SAR BL, 1 ; BL = 00100110b, CF=0.

RET
OF=0 if first operand keeps original sign.

SBB {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}
Subtract with Borrow.
operand1 = operand1 - operand2 - CF

Example 5:87

STC
MOV AL, 5
SBB AL, 3 ; AL = 5 - 3 - 1 = 1

RET

SHL {memory, immediate or REG, immediate or memory, CL or REG, CL}
Shift operand1 Left. The number of shifts is set by operand2.

Example 5:88

MOV AL, 11100000b
SHL AL, 1 ; AL = 11000000b, CF=1.

RET
OF=0 if first operand keeps original sign.

SHR {memory, immediate or REG, immediate or memory, CL or REG, CL}
Shift operand1 Right. The number of shifts is set by operand2.

Example 5:89

MOV AL, 00000111b
SHR AL, 1 ; AL = 00000011b, CF=1.

RET
OF=0 if first operand keeps original sign.

STC Set Carry flag.
CF = 1

STD Set Direction flag.
SI and DI will be decremented by chain instructions: CMPSB, CMPSW, LODSB, LODSW, MOVSb, MOVSW, STOSB, STOSW.
DF = 1

STI Set Interrupt enable flag.
This enables hardware interrupts.
IF = 1

STOSB Store byte in AL into ES:[DI]. Update DI.

ES:[DI] = AL
if DF = 0 then DI = DI + 1 else DI = DI - 1

Example 5:90

```
LEA DI, a1
MOV AL, 12h
MOV CX, 5
```

```
REP STOSB
```

```
RET
```

```
a1 DB 5 dup(0)
```

STOSW Store word in AX into ES:[DI]. Update DI.
ES:[DI] = AX
if DF = 0 then DI = DI + 2 else DI = DI - 2

Example 5:91

```
LEA DI, a1
MOV AX, 1234h
MOV CX, 5
REP STOSW
RET
a1 DW 5 dup(0)
```

SUB {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}
Subtract.
operand1 = operand1 - operand2

Example 5:92

```
MOV AL, 5
SUB AL, 1 ; AL = 4
RET
```

TEST {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}
Logical AND between all bits of two operands for flags only. These flags are affected: **ZF**, **SF**, **PF**. Result is not stored anywhere.

Example 5:93

```
MOV AL, 00000101b
TEST AL, 1      ; ZF = 0.
TEST AL, 10b    ; ZF = 1.
RET
```

XCHG {REG, memory or memory, REG or REG, REG}

Exchange values of two operands.

operand1 < - > operand2

Example 5:94

```
MOV AL, 5
MOV AH, 2
XCHG AL, AH ; AL = 2, AH = 5
XCHG AL, AH ; AL = 5, AH = 2
RET
```

XLATB Translate byte from table.

Copy value of memory byte at DS:[BX + unsigned AL] to AL register.

AL = DS:[BX + unsigned AL]

Example 5:95

```
ORG 100h
LEA BX, dat
MOV AL, 2
XLATB ; AL = 33h
```

RET

dat DB 11h, 22h, 33h, 44h, 55h

XOR {REG, memory or memory, REG or REG, REG or memory, immediate or REG, immediate}

Logical XOR (Exclusive OR) between all bits of two operands. Result is stored in first operand.

Example 5:96

```
MOV AL, 00000111b
XOR AL, 00000010b ; AL = 00000101b
RET
```

Review Questions

1. Give examples (two from each) of the different data movement instructions possible involving mnemonic **MOV**.
2. Name the conditional jumps in 8086.
3. What are the limitations of conditional jumps?
4. Explain with examples adjustments needed for arithmetic involving ASCII numbers.
5. Explain the usage of ADD and ADC instructions.
6. Explain the usage of SUB and SBB instructions.
7. Explain the usage of MUL and IMUL instructions.
8. Explain the usage of DIV and IDIV instructions.
9. Explain how the XLATB instruction is helpful for database handling.